

第一章 绪论

算法：无

第一章节中内容没有算法，着重记忆名词概念，优缺点，作用

第二章 线性表

~~基本操作~~

~~不必全部记忆，挑典型的基本操作算法~~

~~InitList(&L)~~

~~操作结果：构造一个空的线性表 L。~~

~~DestroyList(&L)~~

~~初始条件：线性表 L 已存在。~~

~~操作结果：销毁线性表 L。~~

~~ClearList(&L)~~

~~初始条件：线性表 L 已存在。~~

~~操作结果：将 L 重置为空表。~~

~~ListEmpty(L)~~

~~初始条件：线性表 L 已存在。~~

~~操作结果：若 L 为空表，则返回 true，否则返回 false。~~

~~ListLength(L)~~

~~初始条件：线性表 L 已存在。~~

~~操作结果：返回 L 中数据元素的个数。~~

~~GetElem(L, i, &e)~~

~~初始条件：线性表 L 已存在，且 $1 \leq i \leq \text{ListLength}(L)$ 。~~

~~操作结果：用 e 返回 L 中第 i 个数据元素的值。~~

~~LocateElem(L, e)~~

~~初始条件：线性表 L 已存在。~~

~~操作结果：返回 L 中第 1 个值与 e 相同的元素在 L 中的位置。若这样的数据元素不存在，则返回值为 0。~~

第三章 栈和队列

一些基本操作需要记忆（InitQueue(&Q)操作结果：构造一个空队列Q。
DestroyQueue(&Q)初始条件：队列Q已存在。操作结果：队列Q被销毁，不再存在……）

算法：数制的转换

【算法步骤】

- ① 初始化空栈S。
- ② 当十进制数N非零时，循环执行以下操作：
把N与8求余得到的八进制数压入栈S；
N更新为N与8的商。
- ③ 当栈S非空时，循环执行以下操作：
弹出栈顶元素e；
输出e。

```
void conversion(int N)
{
    // 对于任意一个非负十进制数，打印输出与其等值的八进制数
    InitStack(S); // 初始化空栈S
    while(N)      // 当N非零时，循环
    {
        Push(S, N%8); // 把N与8求余得到的八进制数压入栈S
        N=N/8;        // N更新为N与8的商
    }
    while(!StackEmpty(S)) // 当栈S非空时，循环
    {
        Pop(S, e); // 弹出栈顶元素e
        cout<<e;  // 输出e
    }
}
```

该算法的时间和空间复杂度均为 $O(\log_8 n)$ 。
作答时可以写一个具体的例子（简单的就可以）

算法 括号匹配

【算法步骤】

背诵前，先手推一遍执行逻辑，然后开始写代码，写完代码照着代码把文字描述写一遍
然后把时空复杂度写下

```
bool Matching (char str[], int length) {  
    SqStack S;  
    InitStack(S); //初始化一个栈  
    for (int i=0; i<length; i++){  
        if (str[i]=='(' || str[i]=='[' || str[i]=='{'){  
            Push(S, str[i]); //扫描到左括号，入栈  
        } else {  
            if (StackEmpty(S)) //扫描到右括号，且当前栈空  
                return false; //匹配失败  
  
            char topElem;  
            Pop(S, topElem); //栈顶元素出栈  
            if(str[i]==')' && topElem!='(')  
                return false;  
            if(str[i]==']' && topElem!='[')  
                return false;  
            if(str[i]=='}' && topElem!='{')  
                return false;  
        }  
    }  
    return StackEmpty(S); //检索完全部括号后，栈空说明匹配成功  
}
```

#define MaxSize 10
typedef struct{
 char data[MaxSize];
 int top;
} SqStack;

//定义栈中元素的最大个
//静态数组存放栈中元素
//栈顶指针

初始化顺序栈：定义顺序栈S，创建一个空的顺序栈。

遍历字符数组：用循环逐个扫描str的每个字符：

遇到左括号时：若当前字符是左括号([{ 三者之一，调用入栈操作，将该左括号压入栈中。

遇到右括号时：先调用栈空判断操作判定栈是否为空：

若栈为空：说明该右括号无对应左括号，匹配失败，返回false；

若栈非空：调用出栈操作，取出栈顶元素，检查是否和当前括号匹配：

若当前是)但topElem不是(，返回false；

若当前是]但topElem不是[，返回false；

若当前是}但topElem不是{，返回false。

遍历结束后判栈空：字符全部扫描完后，调用StackEmpty(S)：

若栈为空：所有左括号都匹配了右括号，返回true；

若栈非空：存在未匹配的左括号，返回false。

此算法要从头到尾查找表达式中每个字符，若表达式的字符串长度为n，则此算法的时间复杂度为O(n)。空间复杂度也同为O(n)。

算法 表达式求值

【算法步骤】

- ① 初始化 OPTR 栈和 OPND 栈，将表达式起始符 “#” 压入 OPTR 栈。
- ② 查找表达式，读入第一个字符 ch，如果表达式没有查找完毕至 “#” 或 OPTR 的栈顶元素不为 “#” 时，则循环执行以下操作。
 - 若 ch 不是运算符，则压入 OPND 栈，读入下一字符 ch。
 - 若 ch 是运算符，则根据 OPTR 的栈顶元素和 ch 的优先级比较结果，做不同的处理：
 - 若小于，则将 ch 压入 OPTR 栈，读入下一字符 ch；
 - 若大于，则弹出 OPTR 栈顶的运算符，从 OPND 栈弹出两个数，进行相应运算，将结果压入 OPND 栈；
 - 若等于，则 OPTR 的栈顶元素是 “(” 且 ch 是 “)”，这时弹出 OPTR 栈顶的 “(”，相当于括号匹配成功，然后读入下一字符 ch。
- ③ OPND 栈顶元素即表达式求值结果，返回此元素。

```
char EvaluateExpression()  
{// 算术表达式求值的算符优先算法，设 OPTR 和 OPND 分别为运算符栈和操作数栈  
    InitStack(OPND); // 初始化 OPND 栈  
    InitStack(OPTR); // 初始化 OPTR 栈  
    Push(OPTR, '#'); // 将表达式起始符 “#” 压入 OPTR 栈  
    cin >> ch;  
    while(ch != '#' || GetTop(OPTR) != '#') // 表达式没有查找完毕或 OPTR 的栈顶元素不为 “#”  
    {  
        if(!In(ch)) {Push(OPND, ch); cin >> ch;} // ch 不是运算符则进 OPND 栈  
        else  
            switch(Precede(GetTop(OPTR), ch)) // 比较 OPTR 的栈顶元素和 ch 的优先级  
            {  
                case '<':  
                    Push(OPTR, ch); cin >> ch; // 当前字符 ch 压入 OPTR 栈，读入下一字符 ch  
                    break;  
                case '>':  
                    Pop(OPTR, theta); // 弹出 OPTR 栈顶的运算符  
                    Pop(OPND, b); Pop(OPND, a); // 弹出 OPND 栈顶的两个运算数  
                    Push(OPND, Operate(a, theta, b)); // 将运算结果压入 OPND 栈  
                    break;  
                case '=':  
                    Pop(OPTR, x); cin >> ch; // OPTR 的栈顶元素是 “(” 且 ch 是 “)”  
                    // 弹出 OPTR 栈顶的 “(”，读入下一字符 ch  
                    break;  
            } // switch  
    } // while  
    return GetTop(OPND); // OPND 栈顶元素即表达式求值结果  
}
```

In() 是判定读入的字符 ch 是否为运算符的函数

Operate() 是进行二元运算的函数。Precede() 是判定运算符栈的栈顶元素与读入的运算符之间优先关系的函数

此算法从头到尾查找表达式中每个字符，若表达式的字符串长度为 n，则

此算法的时间复杂度为 $O(n)$

算法在运行时所占用的辅助空间主要取决于 OPTR 栈和 OPND 栈的大小，显然，它

们的空间大小之和不会超过 n ，所以此算法的空间复杂度也同样为 $O(n)$ 。
考试也可写一个简单的例子

算法 舞伴问题

【算法步骤】

- ① 初始化 Mdancers 队列和 Fdancers 队列。
- ② 反复循环，依次将跳舞者姓名根据性别插入 Mdancers 队列或 Fdancers 队列。
- ③ 当 Mdancers 队列和 Fdancers 队列均为非空时，反复循环，依次输出男女舞伴的姓名。
- ④ 如果 Mdancers 队列为空而 Fdancers 队列非空，则输出 Fdancers 队列的队头女士的姓名。
- ⑤ 如果 Fdancers 队列为空而 Mdancers 队列非空，则输出 Mdancers 队列的队头男士的姓名。

```
void DancePartner(Person dancer[],int num)
{
    // 结构数组 dancer 中存放跳舞的男女姓名，num 是跳舞的人数。
    InitQueue(Mdancers); // 男士队列初始化
    InitQueue(Fdancers); // 女士队列初始化
    for(i=0;i<num;i++) // 依次将跳舞者根据其性别入队
    {
        p=dancer[i];
        if(p.sex=='F') EnQueue(Fdancers,p); // 插入女队
        else EnQueue(Mdancers,p); // 插入男队
    }
    cout<<"The dancing partners are:\n";
    while(!QueueEmpty(Fdancers) && !QueueEmpty(Mdancers))
    {
        // 依次输出男女舞伴的姓名
        DeQueue(Fdancers,p); // 女士出队
        cout<<p.name<<" "; // 输出出队女士姓名
        DeQueue(Mdancers,p); // 男士出队
        cout<<p.name<<endl; // 输出出队男士姓名
    }
    if(!QueueEmpty(Fdancers)) // 女士队列非空，输出队头女士的姓名
    {
        p=GetHead(Fdancers); // 取女士队头
        cout<<"The first woman to get a partner is: "<< p.name<<endl;
    }
    else if(!QueueEmpty(Mdancers)) // 男士队列非空，输出队头男士的姓名
    {
        p=GetHead(Mdancers); // 取男士队头
        cout<<"The first man to get a partner is: "<< p.name<<endl;
    }
}
```

若跳舞者人数总计为 n ，则此算法的时间复杂度为 $O(n)$ 。空间复杂度取决于 Mdancers 队列和 Fdancers 队列的长度，二者长度之和不会超过 n ，因此空间复杂度也同样为 $O(n)$

记号-PDF

串、数组和广义表

一些基本操作熟记典型的

算法 BF 算法

典型算法 BF 算法，最简单直观的模式匹配算法是 BF (Brute-Force) 算法

子串的定位运算通常称为串的模式匹配或串匹配。

思路：设有两个字符串 S 和 T，设 S 为主串，也称正文串；设 T 为子串，也称为模式。在主串 S 中查找与模式 T 相匹配的子串，如果匹配成功，确定相匹配的子串中的第一个字符在主串 S 中出现的位置。

模式匹配不一定是从主串的第一个位置开始，可以指定主串中查找的起始位置 pos。如果采用

字符串顺序存储结构，可以写出不依赖于其他串操作的匹配算法。

【算法步骤】

① 分别利用计数指针 i 和 j 指示主串 S 和模式 T 中当前正待比较的字符位置，i 初值为 pos，j 初值为 1。

② 如果两个串均未比较到串尾，即 i 和 j 均分别小于等于 S 和 T 的长度时，则循环执行以下操作：

S.ch[i] 和 T.ch[j] 比较，若相等，则 i 和 j 分别指示串中下个位置，继续比较后续字符；

若不相等，指针后退重新开始匹配，从主串的下一个字符 ($i=i-j+2$)

起再重新和模式的第一个字符 ($j=1$) 比较。

③ 如果 $j > T.length$ ，说明模式 T 中的每个字符依次和主串 S 中的一个连续的字符序列相等，则匹配成功，返回和模式 T 中第一个字符相等的字符在主串 S 中的序号 ($i-T.length$)；否则称匹配不成功，返回 0。

```
int Index_BF(SString S, SString T, int pos)
{ // 返回模式 T 在主串 S 中第 pos 个字符开始第一次出现的位置。若不存在，则返回 0
  // 其中，T 非空， $1 \leq pos \leq S.length$ 
  i=pos; j=1; // 初始化
  while(i ≤ S.length && j ≤ T.length) // 两个串均未比较到串尾
  {
    if(S.ch[i]==T.ch[j]){++i;++j;} // 继续比较后继字符
    else{i=i-j+2;j=1;} // 指针后退，重新开始匹配
  }
  if(j > T.length) return i-T.length; // 匹配成功
  else return 0; // 匹配失败
}
```

BF 算法思路直观简明。但当匹配失败时，主串的指针 i 总是回溯到 $i-j+2$ 位置，

模式串的指针总是回溯到首字符位置 $j=1$ ，因此，算法时间复杂度高。

最好情况下的平均时间复杂度是 $O(n+m)$ 。最坏情况下的平均时间复杂度是 $O(n \times m)$ 。

典型例子

第一趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b
a	b	c										

$\downarrow i=3$
 $\uparrow j=3$

第二趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b
a												

$\downarrow i=2$
 $\uparrow j=1$

第三趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b

$\downarrow i=7$
 $\uparrow j=5$

第四趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b

$\downarrow i=4$
 $\uparrow j=1$

第五趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b

$\downarrow i=5$
 $\uparrow j=1$

第六趟匹配

a	b	a	b	c	a	b	c	a	c	b	a	b

$\downarrow i=11$
 $\uparrow j=6$

算法 kmp 算法

此算法可以在 $O(n + m)$ 的时间数量级上完成串的模式匹配操作。

此算法可以在 $O(n + m)$ 的时间数量级上完成串的模式匹配操作。其改进在于：每当一趟匹配过程中出现字符不等时，无须回溯主串的指针，而是利用已经得到的“部分匹配”的结果将模式向右“滑动”尽可能远的一段距离后，继续进行匹配。

- 将主串的第 pos 个字符和模式串的第一个字符比较，
 - 若相等，继续逐个比较后续字符；
 - 若不等，从主串的下一字符起，重新与模式串的第一个字符比较。
- 直到主串的一个连续子串字符序列与模式串相等。返回值为 S 中与 T 匹配的子序列第一个字符的序号，即匹配成功。
- 否则，匹配失败，返回值 0

Kmp 算法描述

```
int Index_KMP(SString S, SString T, int pos)
{ // 利用模式串 T 的 next 函数求 T 在主串 S 中第 pos 个字符之后的位置
  // 其中, T 非空,  $1 \leq pos \leq S.length$ 
  i = pos; j = 1;
  while (i <= S.length && j <= T.length) // 两个串均未比较到串尾
  {
    if (j == 0 || S.ch[i] == T.ch[j]) { ++i; ++j; } // 继续比较后续字符
    else j = next[j]; // 模式串向右移动
  }
  if (j > T.length) return i - T.length; // 匹配成功
  else return 0; // 匹配失败
}
```

Next 数组算法描述

```
void get_next(SString T, int next[])
{ // 求模式串 T 的 next 函数值并将其存入数组 next
  i = 1; next[1] = 0; j = 0;
  while (i < T.length)
  {
    if (j == 0 || T.ch[i] == T.ch[j]) { ++i; ++j; next[i] = j; }
    else j = next[j];
  }
}
```


22-PDF
~~算法 随机稀疏矩阵的压缩存储~~

```
typedef struct {  
    int row, col, value;  
} Triple;  
  
// 稀疏矩阵结构  
typedef struct {  
    Triple data[MAX_SIZE];  
    int rows, cols, count;  
} SparseMatrix;  
  
// 初始化矩阵  
void initMatrix(SparseMatrix *mat, int rows, int cols) {  
    mat->rows = rows;  
    mat->cols = cols;  
    mat->count = 0;  
}  
  
// 存储非零元素  
void storeElement(SparseMatrix *mat, int row, int col, int value) {  
    if (value != 0) {  
        if (mat->count < MAX_SIZE) {  
            mat->data[mat->count].row = row;  
            mat->data[mat->count].col = col;  
            mat->data[mat->count].value = value;  
            mat->count++;  
        }  
    }  
}
```

时间复杂度: $O(k)$ - 存储 k 个非零元素

空间复杂度: $O(k)$ - 与稀疏矩阵的非零元素数量成正比

算法 稀疏矩阵的一次定位快速转置算法

核心思路

稀疏矩阵一次定位快速转置算法的核心思想在于通过预处理实现精准定位。该算法首先统计原矩阵每列的非零元素个数，进而计算出转置后每行元素在结果数组中的起始位置，最终通过一次遍历即可将每个元素直接放置到预定位置。

```
typedef struct {
    int row;    // 非零元素所在的行号
    int col;    // 非零元素所在的列号
    int value;  // 非零元素的值
} Triple;

void fastTranspose(const Triple* src, Triple* dest, int rows, int cols,
int terms) {
    if (terms <= 0) return; // 如果没有非零元素，直接返回

    // 步骤 1: 计算每列非零元素个数
    // 分配一个大小为 cols 的数组，用于记录原矩阵每列有多少个非零元素
    int* colCount = (int*)calloc(cols, sizeof(int));
    // 遍历所有非零元素，统计每列的非零元素个数
    for (int i = 0; i < terms; i++) {
        colCount[src[i].col]++; // 根据当前元素的列号，增加对应列的计
        数器
    }

    // 步骤 2: 计算每列起始位置
    // 分配一个大小为 cols 的数组，用于记录转置后每行（原矩阵的列）的起
    始位置
    int* startPos = (int*)malloc(cols * sizeof(int));
    startPos[0] = 0; // 第 0 列（转置后第 0 行）的起始位置为 0
    // 计算每列的起始位置：当前列起始位置 = 前一列起始位置 + 前一列非
    零元素个数
    for (int i = 1; i < cols; i++) {
        startPos[i] = startPos[i-1] + colCount[i-1];
    }

    // 步骤 3: 元素转置
    // 遍历原矩阵的所有非零元素，进行转置操作
    for (int i = 0; i < terms; i++) {
        // 根据当前元素的列号，找到在转置矩阵中的存放位置
        int pos = startPos[src[i].col];

        // 执行转置：行列互换
        dest[pos].row = src[i].col; // 原列号变为转置后的行号
        dest[pos].col = src[i].row; // 原行号变为转置后的列号
    }
}
```

```
dest[pos].value = src[i].value; // 值保持不变
// 更新起始位置，为下一个相同列号的元素准备位置
startPos[src[i].col]++;
}
```

该快速转置算法的时间复杂度为 $O(\text{cols} + \text{terms})$ ，其中 cols 为矩阵列数， terms 为非零元素个数，这是因为算法需要分别遍历列数和非零元素各一次；空间复杂度为 $O(\text{cols})$ ，主要来自于存储每列非零元素个数和起始位置的两个辅助数组。这种复杂度表现使得算法特别适合处理大规模稀疏矩阵

见8-PDF

第六章 树与二叉树

一些基本操作记忆一下，记忆典型的即可，因为在后面的算法中可能会调用到基本操作。

算法 二叉树的遍历

先序/中序/后序遍历算法

先序遍历二叉树的操作定义如下：

若二叉树为空，则操作为空；否则

- (1) 访问根节点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树。

中序遍历二叉树的操作定义如下：

若二叉树为空，则操作为空；否则

- (1) 中序遍历左子树；
- (2) 访问根节点；
- (3) 中序遍历右子树。

117

后序遍历二叉树的操作定义如下：

若二叉树为空，则操作为空；否则

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根节点。

二叉树先序遍历算法

```
Status PreOrderTraverse(BiTree T){  
    if(T==NULL) return OK; //空二叉树  
    else{  
        visit(T); //访问根结点  
        PreOrderTraverse(T->lchild); //递归遍历左子树  
        PreOrderTraverse(T->rchild); //递归遍历右子树  
    }  
}
```

中序遍历算法

```
Status InOrderTraverse(BiTree T){  
    if(T==NULL) return OK; //空二叉树  
    else{  
        InOrderTraverse(T->lchild); //递归遍历左子树  
        visit(T); //访问根结点;  
        InOrderTraverse(T->rchild); //递归遍历右子树  
    }  
}
```

后序遍历算法

```
Status PostOrderTraverse(BiTree T){  
    if(T==NULL) return OK; //空二叉树  
    else{  
        PostOrderTraverse(T->lchild); //递归遍历左子树  
        PostOrderTraverse(T->rchild); //递归遍历右子树  
        visit(T); //访问根结点  
    }  
}
```


算法 中序遍历的非递归算法

【算法步骤】

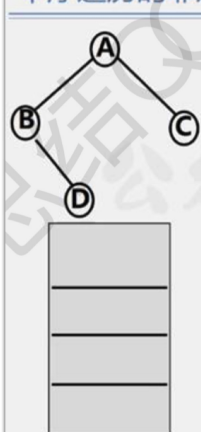
- ① 初始化一个空栈 S ，指针 p 指向根节点。
- ② 申请一个节点空间 q ，用来存放栈顶弹出的元素。
- ③ 当 p 非空或者栈 S 非空时，循环执行以下操作：
如果 p 非空，则使 p 进栈， p 指向该节点的左孩子；
如果 p 为空，则弹出栈顶元素并访问根节点，将 p 指向该节点的右孩子。

```
void InOrder(Tree t) {
    Stack S;
    InitStack(S);

    Tree p;
    p = t;

    while (p || !StackEmpty(S)) {
        if (p) {
            Push(S, p);
            p = p->lchild;
        } else {
            Pop(S, p);
            visit(p);
            p = p->rchild;
        }
    }
}
```

中序遍历的非递归算法 (算法5.2)



```
void PreOrder(Tree t) {
    Stack S;
    InitStack(S);

    Tree p;
    p = t;

    while (p || !StackEmpty(S)) {
        if (p) {
            visit(p);
            Push(S, p);
            p = p->lchild;
        } else {
            Pop(S, p);
            p = p->rchild;
        }
    }
}
```

广度遍历

```
void LevelOrder(Tree t) {
    // 初始化队列
    LinkQueue Q;
    InitQueue(Q);

    // 暂存树结点的指针
    Tree p;

    // 先把根结点入队
    EnQueue(Q, t);

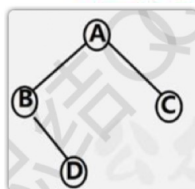
    // 不为空就一直循环
    while (!QueueEmpty(Q)) {
        // 出队并且打印 该树结点
        DeQueue(Q, p);
        visit(p);
        // printf("%c", p->data);

        // 如果左孩子结点不为空就入队
        if (p->lchild != NULL) {
            EnQueue(Q, p->lchild);
        }

        // 如果右孩子结点不为空就入队
        if (p->rchild != NULL) {
            EnQueue(Q, p->rchild);
        }
    }
}
```

```
void LevelOrder(Tree T) {
    InitQueue(Q);
    Tree p;
    p=T;

    EnQueue(Q, T);
    while (!EmptyQueue(Q)) {
        DeQueue(Q, p);
        visit(p);
        if (p->lchild) {
            EnQueue(Q, p->lchild);
        }
        if (p->rchild) {
            EnQueue(Q, p->rchild);
        }
    }
}
```



算法 按照先序遍历的顺序建立二叉链表

【算法步骤】

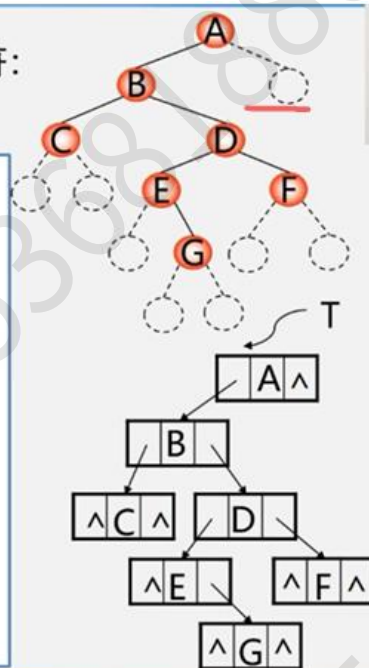
- ① 查找字符序列，读入字符 ch。
- ② 如果 ch 是一个“#”字符，则表明该二叉树为空树，即 T 为 NULL；否则执行以下操作：

申请一个节点空间 T；
将 ch 赋给 T->data；
递归创建 T 的左子树；
递归创建 T 的右子树。

对右图所示二叉树，按下列顺序读入字符：

ABC##DE##G##F###

```
Status CreateBiTree(BiTree &T) {  
    scanf(&ch); //cin>>ch;  
    if (ch == "#") T = NULL;  
    else {  
        if (!(T = (BiTNode *)malloc(sizeof(BiTNode))))  
            exit(OVERFLOW); //T=new BiTNode;  
        T->data = ch; // 生成根结点  
        CreateBiTree(T->lchild); // 构造左子树  
        CreateBiTree(T->rchild); // 构造右子树  
    }  
    return OK;  
} // CreateBiTree
```



算法 复制二叉树

搞不懂逻辑，简单看看，能背住就背住

【算法步骤】

如果是空树，递归结束，否则执行以下操作：

申请一个新节点空间，复制根节点；
递归复制左子树；
递归复制右子树。

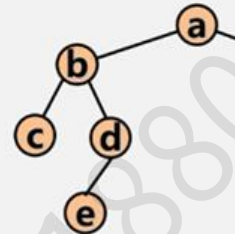
```
void Copy(BiTree T, BiTree &NewT)  
{ // 复制一棵和 T 完全相同的二叉树  
    if (T == NULL) // 如果是空树，递归结束  
    {  
        NewT = NULL;  
        return;  
    }  
    else  
    {  
        NewT = new BiTNode;  
        NewT->data = T->data; // 复制根节点  
        Copy(T->lchild, NewT->lchild); // 递归复制左子树  
        Copy(T->rchild, NewT->rchild); // 递归复制右子树  
    }  
}
```

28-PDF

二叉树遍历算法的应用——计算二叉树深度 (算法5.5)

- 如果是空树，则深度为0；
- 否则，递归计算左子树的深度记为 m ，递归计算右子树的深度记为 n ，二叉树的深度则为 m 与 n 的较大者加1。

```
int Depth( BiTree T){  
    if(T==NULL)    return 0; //如果是空树返回0  
    else {  
        m=Depth(T->lChild);  
        n =Depth(T->rChild);  
        if(m>n) return (m+1);  
        else return(n+1);  
    }  
}
```

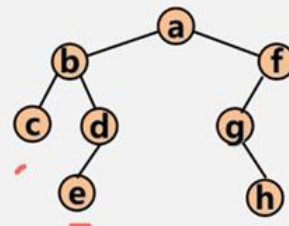


二叉树遍历算法的应用

——计算二叉树叶子结点数 (补充)

- 如果是空树，则叶子结点个数为0；
- 否则，为左子树的叶子结点个数+右子树的叶子结点个数。

```
int LeafCount(BiTree T){  
    if(T==NULL)    //如果是空树返回0  
        return 0;  
    if (T->lchild == NULL && T->rchild == NULL)  
        return 1; //如果是叶子结点返回1  
    else  
        return LeafCount(T->lchild) +  
            LeafCount(T->rchild);  
}
```

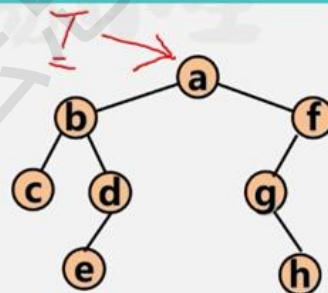


计算二叉树结点总数二叉树遍历算法的应用

——计算二叉树结点总数 (算法5.6)

- 如果是空树，则结点个数为0；
- 否则，结点个数为左子树的结点个数+右子树的结点个数再+1。

```
int NodeCount(BiTree T){  
    if(T == NULL )  
        return 0;  
    else  
        return NodeCount(T->lchild)+  
            NodeCount(T->rchild)+1;  
}
```



算法 中序遍历线索二叉树

核心思想中序遍历时对当前节点线索化

① 查找 p 指针所指节点的前驱：

若 $p \rightarrow \text{LTag}$ 为 1，则 p 的左链指示其前驱；

若 $p \rightarrow \text{LTag}$ 为 0，则说明 p 有左子树，节点的前驱是遍历左子树时最后访问的一个节点

（左子树中最右下的节点）。

② 查找 p 指针所指节点的后继：

若 $p \rightarrow \text{RTag}$ 为 1，则 p 的右链指示其后继，以图 5.16 所示的中序线索树为例，节点 b 的后继

为节点*；

若 $p \rightarrow \text{RTag}$ 为 0，则说明 p 有右子树。根据中序遍历的规律可知，节点的后继应是遍历其

右子树时访问的第一个节点，即右子树中最左下的节点。例如在找节点*的后继时，首先沿右指针

找到其右子树的根节点-，然后顺其左指针往下直至其左标志为 1 的节点为节点*的后继，在图 5.16

中是节点 c。

初步建成的树，
 $\text{ltag}, \text{rtag}=0$

中序线索化

中国大学MOOC

```
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag; // 左、右线索标志
} ThreadNode, * ThreadTree;

// 全局变量 pre，指向当前访问结点的前驱
ThreadNode *pre=NULL;

// 中序线索化二叉树T
void CreateInThread(ThreadTree T){
    pre=NULL; // pre初始为NULL
    if(T!=NULL){ // 非空二叉树才能线索化
        InThread(T); // 中序线索化二叉树
        if(pre->rchild==NULL)
            pre->rtag=1; // 处理遍历的最后一个结点
    }
}

// 线索二叉树结点
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag; // 左、右线索标志
} ThreadNode, * ThreadTree;
```

// 中序遍历二叉树，一边遍历一边线索化

```
void InThread(ThreadTree T){
    if(T!=NULL){
        InThread(T->lchild); // 中序遍历左子树
        visit(T); // 访问根节点
        InThread(T->rchild); // 中序遍历右子树
    }
}
```

```
void visit(ThreadNode *q) {
    if(q->lchild==NULL){ // 左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL && pre->rchild==NULL){
        pre->rchild=q; // 建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```

第 809 页 第 1 章 第 1 节 第 1 页

5.7.2 哈夫曼树的构造算法

哈夫曼算法 (构造哈夫曼树的方法)

(1) 根据 n 个给定的权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的森林 $F = \{T_1, T_2, \dots, T_n\}$, 其中 T_i 只有一个带权为 w_i 的根结点。

• 构造森林全是根

(2) 在 F 中选取两棵根结点的权值最小的树作为左右子树, 构造一棵新的二叉树, 且设置新的二叉树的根结点的权值为其左右子树上根结点的权值之和。

• 选用两小造新树

(3) 在 F 中删除这两棵树, 同时将新得到的二叉树加入森林中。

• 删除两小添新人

(4) 重复 (2) 和 (3), 直到森林中只有一棵树为止, 这棵树即为哈夫曼树。

哈夫曼树构造算法的实现 (算法5.10)

1. 初始化HT $[1.....2n-1]$: $lch=rch=parent=0$;
2. 输入初始 n 个叶子结点: 置HT $[1.....n]$ 的 weight 值;
3. 进行以下 $n-1$ 次合并, 依次产生 $n-1$ 个结点HT $[i]$, $i=n+1.....2n-1$:
 - a) 在HT $[1..i-1]$ 中选两个 未被选过 (从 $parent == 0$ 的结点中选) 的 weight 最小的两个结点HT $[s1]$ 和 HT $[s2]$, $s1$ 、 $s2$ 为两个最小结点下标;
 - b) 修改HT $[s1]$ 和 HT $[s2]$ 的 parent 值: HT $[s1].parent=i$; HT $[s2].parent=i$;
 - c) 修改新产生的HT $[i]$:
 - HT $[i].weight = HT[s1].weight + HT[s2].weight$;
 - HT $[i].lch = s1$; HT $[i].rch = s2$;


```

void CreatHuffmanTree (HuffmanTree HT, int n){ //构造哈夫曼树——哈夫曼算法
    if(n<=1)return;
    m=2*n-1; //数组共2n-1个元素
    HT=new HTreeNode[m+1]; //0号单元未用, HT[m]表示根结点
    for(i=1;i<=m;++i) { //将2n-1个元素的lch、rch、parent置为0
        HT[i].lch=0; HT[i].rch=0; HT[i].parent=0;
    }
    for(i=1;i<=n;++i) cin>>HT[i].weight; //输入前n个元素的weight值
    //初始化结束, 下面开始建立哈夫曼树
}

```

【算法5.10】续

//续算法5.10

```

for( i=n+1;i<=m; i++) { //合并产生n-1个结点——构造Huffman树
    Select(HT, i-1, s1, s2); //在HT[k](1≤k≤i-1)中选择两个其双亲域为0,
    //且权值最小的结点,并返回它们在HT中的序号s1和s2
    HT[s1].parent=i; HT[s2].parent=i; //表示从F中删除s1,s2
    HT[i].lch=s1; HT[i].rch=s2; //s1,s2分别作为i的左右孩子
    HT[i].weight=HT[s1].weight + HT[s2].weight; //i 的权值为左右孩子权值之和
}
}

```

算法 哈夫曼编码

【算法步骤】

① 分配存储 n 个字符编码的编码表空间 HC ，长度为 $n+1$ ；分配临时存储每个字符编码的动态数组空间 cd ， $cd[n-1]$ 置为 $'\0'$ 。

② 逐个求解 n 个字符的编码，循环 n 次，执行以下操作：

设置变量 $start$ 用于记录编码在 cd 中存放的位置， $start$ 初始时指向最后，即编码结束符位

置 $n-1$ ；

设置变量 c 用于记录从叶子节点向上回溯至根节点所经过的节点下标， c 初始时为当前待

编码字符的下标 i ， f 用于记录 i 的双亲节点的下标；

从叶子节点向上回溯至根节点，求得字符 i 的编码，当 f 没有到达根节点时，循环执行以

下操作：

回溯一次 $start$ 向前指一个位置，即 $--start$ ；

若节点 c 是 f 的左孩子，则生成代码 0，否则生成代码 1，生成的代码 0 或 1 保存在 $cd[start]$ 中；

继续向上回溯，改变 c 和 f 的值。

根据数组 cd 的字符串长度为第 i 个字符编码分配空间 $HC[i]$ ，然后将数组 cd 中的编码复制到 $HC[i]$ 中。

③ 释放临时空间 cd 。

```

void CreatHuffmanCode(HuffmanTree HT,HuffmanCode &HC,int n)
{ // 从叶子到根逆向求每个字符的哈夫曼编码, 存储在编码表HC中
    HC=new char*[n+1]; // 分配存储n个字符编码的编码表空间
    cd=new char[n]; // 分配临时存放每个字符编码的动态数组空间
    cd[n-1]='\0'; // 编码结束符
    for(i=1;i<=n;++i) // 逐个字符求哈夫曼编码
    {
        start=n-1; // start 开始时指向最后, 即编码结束符位置
        c=i;f=HT[i].parent; // f 指向节点c的双亲节点
        while(f!=0) // 从叶子节点开始向上回溯, 直到根节点
        {
            --start; // 回溯一次 start 向前指一个位置
            if(HT[f].lchild==c) cd[start]='0'; // 节点c是f的左孩子, 则生成代码0
            else cd[start]='1'; // 节点c是f的右孩子, 则生成代码1
            c=f;f=HT[f].parent; // 继续向上回溯
        } // 求出第i个字符的编码
        HC[i]=new char[n-start]; // 为第i个字符编码分配空间
        strcpy(HC[i],&cd[start]); // 将求得的编码从临时空间cd复制到HC的当前行中
    } // for
    delete cd; // 释放临时空间
}

```

(1) 编码

有了字符集的哈夫曼编码表之后, 对数据文件的编码过程是: 依次读入文件中的字符 c , 在哈夫曼编码表 HC 中找到此字符, 将字符 c 转换为编码表中存放的编码串。

(2) 译码

对编码后的文件进行译码的过程必须借助于哈夫曼树。具体过程是: 依次读入文件的二进制码, 从哈夫曼树的根节点 (即 $HT[m]$) 出发, 若当前读入 0, 则走向左孩子, 否则走向右孩子。一旦到达某一叶子 $HT[i]$ 时便译出相应的字符编码 $HC[i]$ 。然后重新从根出发继续译码, 直至文件结束。

第七章 图

一些基本操作熟记典型的即可

~~算法~~ 采用邻接矩阵表示法创建无向网

【算法步骤】

- ① 输入总顶点数和总边数。
- ② 依次输入点的信息并将其存入顶点表中。
- ③ 初始化邻接矩阵，使每个权值初始化为极大值。
- ④ 构造邻接矩阵。依次输入每条边依附的顶点和其权值，确定两个顶点在图中的位置之后，使相应边赋予相应的权值，同时使其对称边赋予相同的权值。

```
Status CreateUDN(AMGraph &G)
{
    // 采用邻接矩阵表示法，创建无向网 G
    cin >> G.vexnum >> G.arcnum;           // 输入总顶点数，总边数
    for (i=0; i<G.vexnum; ++i)             // 依次输入点的信息
        cin >> G.vexs[i];
    for (i=0; i<G.vexnum; ++i)             // 初始化邻接矩阵，边的权值
        for (j=0; j<G.vexnum; ++j)         // 均置为极大值 MaxInt
            G.arcs[i][j] = MaxInt;
    for (k=0; k<G.arcnum; ++k)             // 构造邻接矩阵
    {
        cin >> v1 >> v2 >> w;              // 输入一条边依附的顶点及权值
        i = LocateVex(G, v1); j = LocateVex(G, v2); // 确定 v1 和 v2 在 G 中的位置，即顶点数组的下标
        G.arcs[i][j] = w;                  // 边 <v1, v2> 的权值置为 w
        G.arcs[j][i] = G.arcs[i][j];       // 置 <v1, v2> 的对称边 <v2, v1> 的权值为 w
    }
    return OK;
}
```

该算法的时间复杂度是 $O(n^2)$ 。

(1) 优点

- ① 便于判断两个顶点之间是否有边，即根据 $A[i][j] = 0$ 或 1 来判断。
- ② 便于计算各个顶点的度。对于无向图，邻接矩阵第 i 行元素之和就是顶点 v_i 的度；对于有向图，第 i 行元素之和就是顶点 v_i 的出度，第 i 列元素之和就是顶点 v_i 的入度。

(2) 缺点

- ① 不便于增加和删除顶点。
- ② 不便于统计边的数目，需要查找邻接矩阵所有元素才能统计完毕，时间复杂度为 $O(n^2)$ 。
- ③ 空间复杂度高。如果是有向图， n 个顶点需要 n^2 个单元存储边。如果是无向图，因其邻接矩阵是对称的，所以对规模较大的邻接矩阵可以采用压缩存储的方法。

法，仅存储下三角（或上三角）的元素，这样需要 $n(n-1)/2$ 个单元即可。但无论以何种方式存储，邻接矩阵表示法的空间复杂度均为 $O(n^2)$ ，这对于稀疏图而言尤其浪费空间。

~~算法~~ 采用邻接表表示法创建无向图

~~【算法步骤】~~

- ① 输入总顶点数和总边数。
- ② 依次输入点的信息存入顶点表中，使每个表头节点的指针域初始化为 NULL。
- ③ 创建邻接表。依次输入每条边依附的两个顶点，确定这两个顶点的序号 i 和 j 之后，将此边节点分别插入 v_i 和 v_j 对应的两个边链表的头部。

算法6.2 采用邻接表表示法创建无向网

```
Status CreateUDG(ALGraph &G){ //采用邻接表表示法，创建无向图G
    cin>>G.vexnum>>G.arcnum; //输入总顶点数，总边数
    for(i = 0; i<G.vexnum; ++i){ //输入各点，构造表头结点表
        cin>> G.vertices[i].data; //输入顶点值
        G.vertices[i].firstarc=NULL; //初始化表头结点的指针域
    }//for
    for(k = 0; k<G.arcnum;++k){ //输入各边，构造邻接表
        cin>>v1>>v2; //输入一条边依附的两个顶点
        i = LocateVex(G, v1);
        j = LocateVex(G, v2);
```

(接下页)

算法6.2 采用邻接表表示法创建无向网

(接上页)

```
    p1=new ArcNode; //生成一个新的边结点*p1
    p1->adjvex=j; //邻接点序号为j
    p1->nextarc= G.vertices[i].firstarc;
    G.vertices[i].firstarc=p1; //将新结点*p1插入顶点vi的边表头部
    p2=new ArcNode; //生成另一个对称的新的边结点*p2
    p2->adjvex=i; //邻接点序号为i
    p2->nextarc= G.vertices[j].firstarc;
    G.vertices[j].firstarc=p2; //将新结点*p2插入顶点vj的边表头部
} //for
return OK;
} //CreateUDG
```

该算法的时间复杂度是 $O(n + e)$ 。

(1) 优点

① 便于增加和删除顶点。

② 便于统计边的数目，按顶点表顺序查找所有边表可得到边的数目，时间复杂度为

$O(n + e)$ 。

③ 空间效率高。对于一个具有 n 个顶点、 e 条边的图 G ，若 G 是无向图，则在其邻接表表示中

有 n 个顶点表节点和 $2e$ 个边表节点；若 G 是有向图，则在它的邻接表表示或逆邻接表表示中均有

n 个顶点表节点和 e 个边表节点。因此，邻接表或逆邻接表表示的空间复杂度为 $O(n + e)$ ，适合表

示稀疏图。对于稠密图，考虑到邻接表中要附加链域，因此常采取邻接矩阵表示法。

(2) 缺点

① 不便于判断顶点之间是否有边，要判定 v_i 和 v_j 之间是否有边，就需查找第 i 个边表，最坏

情况下时间复杂度为 $O(n)$ 。

② 不便于计算有向图各个顶点的度。对于无向图，在邻接表表示中顶点 v_i 的度是第 i 个边表

中的节点个数。在有向图的邻接表中，第 i 个边表上的节点个数是顶点 v_i 的出度，但求 v_i 的入度较

困难，需遍历各顶点的边表。若有向图采用逆邻接表表示，则与邻接表表示相反，求顶点的入

度较容易，而求顶点的出度较困难。

算法 深度优先搜索遍历连通图

深度优先搜索 (Depth First Search, DFS) 遍历类似于树的先序遍历，是树的先序遍历的推广。

对于一个连通图，深度优先搜索遍历的过程如下。

- (1) 从图中某个顶点 v 出发，访问 v 。
- (2) 找出刚访问过的顶点的第一个未被访问的邻接点，访问该顶点。以该顶点为新顶点，重复此步骤，直至刚访问过的顶点没有未被访问的邻接点为止。
- (3) 返回前一个访问过的且仍有未被访问的邻接点的顶点，找出该顶点的下一个未被访问的邻接点，访问该顶点。
- (4) 重复步骤 (2) 和步骤 (3)，直至图中所有顶点都被访问过，搜索结束。

【算法步骤】

- ① 从图中某个顶点 v 出发，访问 v ，并置 $visited[v]$ 的值为 $true$ 。
- ② 依次检查 v 的所有邻接点 w ，如果 $visited[w]$ 的值为 $false$ ，再从 w 出发进行递归遍历，直到图中所有顶点都被访问过。

【算法描述】

```
bool visited[MVNum]; // 访问标志数组，其初值为“false”
void DFS(Graph G, int v)
{ // 从第  $v$  个顶点出发递归地深度优先遍历图  $G$ 
    cout << v; visited[v] = true; // 访问第  $v$  个顶点，并置访问标志数组相应分量
    值为 true
    for (w = FirstAdjVex(G, v); w >= 0; w = NextAdjVex(G, v, w))
        // 依次检查  $v$  的所有邻接点  $w$ ，FirstAdjVex(G, v) 表示  $v$  的第一个邻接点
        // NextAdjVex(G, v, w) 表示  $v$  相对于  $w$  的下一个邻接点， $w \geq 0$  表示存在邻接点
        if (!visited[w]) DFS(G, w); // 对  $v$  的尚未访问的邻接顶点  $w$  递归调用 DFS()
    }
```

若是非连通图，上述遍历过程执行之后，图中一定还有顶点未被访问，需要从图中另选一个未被访问的顶点作为起始点，重复上述深度优先搜索过程，直到图中所有顶点均被访问过为止。

X 见另-PDF

算法 广度优先搜索遍历连通图

广度优先搜索 (Breadth First Search, BFS) 遍历类似于树的按层次遍历的过程。

广度优先搜索遍历的过程如下。

- (1) 从图中某个顶点 v 出发, 访问 v 。
- (2) 依次访问 v 的各个未曾访问过的邻接点。
- (3) 分别从这些邻接点出发依次访问它们的邻接点, 并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问。重复步骤 (3), 直至图中所有已被访问的顶点的邻接点都被访问到。

【算法步骤】

- ① 从图中某个顶点 v 出发, 访问 v , 并置 $visited[v]$ 的值为 $true$, 然后使 v 入队。
- ② 只要队列不空, 则重复下述操作:
 - 队头元素 u 出队;
 - 依次检查 u 的所有邻接点 w , 如果 $visited[w]$ 的值为 $false$, 则访问 w , 并置 $visited[w]$ 的值为 $true$, 然后使 w 入队。

广度优先遍历

算法6.7 按广度优先非递归遍历连通图G

```
void BFS (Graph G, int v){ //按广度优先非递归遍历连通图G
    cout<<v; visited[v] = true; //访问第v个顶点
    InitQueue(Q); //辅助队列Q初始化, 置空
    EnQueue(Q, v); //v进队
    while(!QueueEmpty(Q)){ //队列非空
        DeQueue(Q, u); //队头元素出队并置为u
        for(w = FirstAdjVex(G, u); w>=0; w = NextAdjVex(G, u, w))
            if(!visited[w]){ //w为u的尚未访问的邻接顶点
                cout<<w; visited[w] = true; EnQueue(Q, w); //w进队
            } //if
        } //while
    } //BFS
```


算法 图的应用最小生成树

方法有两种，prim 和 Kruskal 先介绍 prim 算法

prim 算法形式有两种。一种是王道上的伪代码简单好理解

另一种是参考书中的完整代码，理解起来比较困难

2025/7/17

Prim 算法

prim 普里姆 王道版

Prim 算法的步骤如下：

假设 $G = (V, E)$ 是连通图，其最小生成树 $T = (U, E_T)$ ， E_T 是最小生成树中边的集合。

初始化：向空树 $T = (U, E_T)$ 中添加图 $G = (V, E)$ 的任意一个顶点 u_0 ，使 $U = \{u_0\}$ ， $E_T = \emptyset$ 。

循环（重复下列操作直至 $U = V$ ）：从图 G 中选择满足 $\{(u, v) | u \in U, v \in V - U\}$ 且具有最小权值的边 (u, v) ，加入树 T ，置 $U = U \cup \{v\}$ ， $E_T = E_T \cup \{(u, v)\}$ 。

Prim 算法的简单实现如下：

```
void Prim(G, T) {  
    T =  $\emptyset$ ; //初始化空树  
    U = {w}; //添加任意一个顶点 w  
    while ((V - U) !=  $\emptyset$ ) { //若树中不含全部顶点  
        设 (u, v) 是使  $u \in U$  与  $v \in (V - U)$ ，且权值最小的边；  
        T = T  $\cup$  { (u, v) }; //边归入树  
        U = U  $\cup$  {v}; //顶点归入树  
    }  
}
```

Prim 算法的时间复杂度为 $O(|V|^2)$ ，不依赖于 $|E|$ ，因此它适用于求解边稠密的图的最小生成树。虽然采用其他方法能改进 Prim 算法的时间复杂度，但增加了实现的复杂性。

参考书版

【算法步骤】

① 首先将初始顶点 u 加入 U 中，对其余的每一个顶点 v_j ，将 $closedge[j]$ 均初始化为到 u 的边信息。

② 循环 $n - 1$ 次，做如下处理：

从各组边 $closedge$ 中选出最小边 $closedge[k]$ ，输出此边；
将 k 加入 U 中；

更新剩余的每组最小边信息 $closedge[j]$ ，对于 $V - U$ 中的边，新增加了一条从 k 到 j 的边，
如果新边的权值比 $closedge[j].lowcost$ 小，则将 $closedge[j].lowcost$ 更新为

新边的权值。

```
struct
{
    VerTexType  adjvex;    //最小边在 U 中的那个顶点
    ArcType     lowcost;   //最小边上的权值
}closededge[MVNum];
```

```
void MiniSpanTree_Prim(AMGraph G, VerTexType u)
{
    // 无向图 G 以邻接矩阵形式存储, 从顶点 u 出发构造 G 的最小生成树 T, 输出 T 的各条边
    k=LocateVex(G,u);           //k 为顶点 u 的下标
    for(j=0;j<G.vexnum;++j)     //对 V-U 的每一个顶点初始化 closededge[j]
        if(j!=k) closededge[j]={u,G.arcs[k][j]};           //{adjvex, lowcost}
    closededge[k].lowcost=0;      //初始, U={u}
    for(i=1;i<G.vexnum;++i)
    {
        // 选择其余 n-1 个顶点, 生成 n-1 条边 (n=G.vexnum)
        k=Min(closededge);
        // 求出 T 的下一个节点: 第 k 个顶点, closededge[k] 中存有当前最小边
        u0=closededge[k].adjvex; //u0 为最小边的一个顶点, u0 ∈ U
        v0=G.vexs[k];           //v0 为最小边的另一个顶点, v0 ∈ V-U
        cout<<u0<<v0;          //输出当前的最小边 (u0, v0)
        closededge[k].lowcost=0; //第 k 个顶点并入 U 集
        for(j=0;j<G.vexnum;++j)
            if(G.arcs[k][j]<closededge[j].lowcost) //新顶点并入 U 后重新选择最小边
                closededge[j]={G.vexs[k],G.arcs[k][j]};
    }
}
```

初始状态时, 由于 $U = \{v_1\}$, 则到 $V - U$ 中各顶点的最小边, 即从依附于顶点 v_1 的各条边中, 找到一条权值最小的边 (v_1, v_3) 为生成树上的第一条边, 同时将顶点 v_3 并入集合 U 中。

~~WJ-PDF~~ Kruskal (克鲁斯卡尔) 算法

王道版

与 Prim 算法从顶点开始扩展最小生成树不同, Kruskal 算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法。

Kruskal 算法的步骤如下:

假设 $G = (V, E)$ 是连通图, 其最小生成树 $T = (U, E_T)$ 。

初始化: $U = V, E_T = \emptyset$ 。即每个顶点构成一棵独立的树, T 此时是一个仅含 $|V|$ 个顶点的森林。

循环 (重复直至 T 是一棵树): 按 G 的边的权值递增顺序依次从 $E - E_T$ 中选择一条边, 若这条边加入 T 后不构成回路, 则将其加入 E_T , 否则舍弃, 直到 E_T 中含有 $n-1$ 条边。

Kruskal 算法的简单实现如下:

```
void Kruskal(V,T){
    T=V;                                     //初始化树 T, 仅含顶点
    numS=n;                                 //连通分量数
    while (numS>1){                          //若连通分量数大于 1
        从 E 中取出权值最小的边 (v,u);
        if(v 和 u 属于 T 中不同的连通分量){
            T=T∪{(v,u)};                   //将此边加入生成树中
            numS--;                          //连通分量数减 1
        }
    }
}
```

在 Kruskal 算法中, 最坏情况需要对 $|E|$ 条边各扫描一次。通常采用堆(见第 7 章)来存放边的集合, 每次选择最小权值的边需要 $O(\log_2 |E|)$ 的时间; 每次使用并查集来快速判断两个顶点是否属于一个集合所需的时间为 $O(\alpha(|V|))$, $\alpha(|V|)$ 的增长极其缓慢, 可视为常数。算法的总时间复杂度为 $O(|E|\log_2 |E|)$, 不依赖于 $|V|$, 因此 Kruskal 算法适合于边稀疏而顶点较多的图。

参考书版

算法的实现要引入以下辅助的数据结构。

① 结构体数组 Edge: 存储边的信息, 包括边的两个顶点信息和权值。

```
// 辅助数组 Edges 的定义
struct
{
    VerTexType Head;           // 边的始点
    VerTexType Tail;           // 边的终点
    ArcType lowcost;            // 边上的权值
}Edge[arcnum];
```

② Vexset[i]: 标识各个顶点所属的连通分量。对每个顶点 $v_i \in V$, 在辅助数组中存在一个相应元素 Vexset[i] 表示该顶点所在的连通分量。初始时 Vexset[i] = i, 表示各顶点自成一个连通分量。

```
// 辅助数组 Vexset 的定义
int Vexset[MVNum];
```

【算法步骤】

- ① 将数组 Edge 中的元素按权值从小到大排序。
- ② 依次查看数组 Edge 中的边，循环执行以下操作：
依次从排好序的数组 Edge 中选出一条边 (v1, v2)；
在 Vexset 中分别查找 v1 和 v2 所在的连通分量 vs1 和 vs2 进行判断：
如果 vs1 和 vs2 不等，表明所选的两个顶点分属不同的连通分量，输出此边，并合并 vs1 和 vs2 两个连通分量；
如果 vs1 和 vs2 相等，表明所选的两个顶点属于同一个连通分量，舍去此边而选择下一条权值最小的边。

```
void MiniSpanTree_Kruskal(AMGraph G)
{ // 无向网 G 以邻接矩阵形式存储, 构造 G 的最小生成树 T, 输出 T 的各条边
    Sort(Edge); // 将数组 Edge 中的元素按权值从小到大排序
    for(i=0; i<G.vexnum; ++i) // 辅助数组, 表示各顶点自成一个连通分量
        Vexset[i]=i;
    for(i=0; i<G.arcnum; ++i) // 依次查看数组 Edge 中的边
    {
        v1=LocateVex(G, Edge[i].Head); // v1 为边的始点 Head 的下标
        v2=LocateVex(G, Edge[i].Tail); // v2 为边的终点 Tail 的下标
        vs1=Vexset[v1]; // 获取边 Edge[i] 的始点所在的连通分量 vs1
        vs2=Vexset[v2]; // 获取边 Edge[i] 的终点所在的连通分量 vs2
        if(vs1!=vs2) // 边的两个顶点分属不同的连通分量
        {
            cout<< Edge[i].Head << Edge[i].Tail; // 输出此边
            for(j=0; j<G.vexnum; ++j) // 合并 vs1 和 vs2 两个分量, 即两个集合统一编号
                if(Vexset[j]==vs2) Vexset[j]=vs1; // 集合编号为 vs2 的都改为 vs1
        } // if
    } // for
}
```

假若以第 8 章将介绍的“堆”来存放网中的边进行堆排序, 对于包含 e 条边的网, 上述算法排序时间是 $O(e \log 2e)$
克鲁斯卡尔算法的时间复杂度为 $O(e \log 2e)$

~~算法-PPJ~~

算法 图的应用最短路径算法

最短路径算法有两种一种是多源最短路径算法另一种是单源最短路径算法

单源最短路径算法——迪杰斯特拉算法 (Dijkstra)

假设用带权的邻接矩阵 arcs 来表示带权有向网 G, $G.arcs[i][j]$ 表示弧 $\langle v_i, v_j \rangle$ 上的权值。若 $\langle v_i, v_j \rangle$ 不存在, 则置 $G.arcs[i][j]$ 为 ∞ , 源点为 v_0 。

算法的实现要引入以下辅助的数据结构。

① 一维数组 $S[i]$: 记录从源点 v_0 到终点 v_i 是否已被确定最短路径长度, true 表示确定, false 表示尚未确定。

② 一维数组 $Path[i]$: 记录从源点 v_0 到终点 v_i 的当前最短路径上 v_i 的直接前驱顶点序号。其初

值为: 如果从 v_0 到 v_i 有弧, 则 $Path[i]$ 为 v_0 , 否则为 -1。

③ 一维数组 $D[i]$: 记录从源点 v_0 到终点 v_i 的当前最短路径长度。其初值为: 如果从 v_0 到 v_i 有

弧, 则 $D[i]$ 为弧上的权值, 否则为 ∞ 。

显然, 最短路径必为 (v_0, v_k) , 其满足以下条件:

$$D[k] = \min\{D[i] \mid v_i \in V - S\}$$

求得顶点 v_k 的最短路径后, 将其加入第一组顶点集 S 中。

每当加入一个新的顶点到顶点集 S , 对第二组剩余的各个顶点而言, 多了一个“中转”顶

点, 从而多了一个“中转”路径, 所以要对第二组剩余的各个顶点的最短路径长度进行更新。

原来 v_0 到 v_i 的最短路径长度为 $D[i]$, 加入 v_k 之后, 以 v_k 作为中间顶点的“中转”路径长度为

$D[k] + G.arcs[k][i]$, 若 $D[k] + G.arcs[k][i] < D[i]$, 则用 $D[k] + G.arcs[k][i]$ 取代 $D[i]$ 。

更新后, 再选择数组 D 中值最小的顶点加入第一组顶点集 S 中, 如此进行下去, 直到图中所

有顶点都加入第一组顶点集 S 中为止。

【算法步骤】

① 初始化:

将源点 v_0 加到 S 中, 即 $S[v_0] = \text{true}$;

将 v_0 到各个终点的最短路径长度初始化为权值, 即 $D[i] = G.arcs[v_0][v_i]$ ($v_i \in V - S$);

如果 v_0 和顶点 v_i 之间有弧, 则将 v_i 的前驱置为 v_0 , 即 $Path[i] = v_0$, 否则

$Path[i] = -1$ 。

② 循环 $n - 1$ 次, 执行以下操作。

选择下一条最短路径的终点 v_k , 使得:

$$D[k] = \min\{D[i] \mid v_i \in V - S\}$$

将 v_k 加到 S 中, 即 $S[v_k] = \text{true}$;

根据条件更新从 v_0 出发到集合 $V - S$ 上任一顶点的最短路径的长度, 若条件 $D[k] + G.arcs$

$[k][i] < D[i]$ 成立, 则更新 $D[i] = D[k] + G.arcs[k][i]$, 同时更改 v_i 的前驱为 v_k , $Path[i] = k$ 。

【算法描述】

```
void ShortestPath_DIJ(AMGraph G, int v0)
{
    // 用Dijkstra算法求有向网的v0顶点到其余顶点的最短路径
    n=G.vexnum; // n为G中顶点的个数
    for(v=0;v<n;++v) // n个顶点依次初始化
    {
        S[v]=false; // S初始为空集
        D[v]=G.arcs[v0][v]; // 将v0到各个终点的最短路径长度初始化为弧上的权值
        if(D[v]<MaxInt) Path[v]=v0; // 如果v0和v之间有弧,则将v的前驱置为v0
        else Path[v]=-1; // 如果v0和v之间无弧,则将v的前驱置为-1
    } // for
    S[v0]=true; // 将v0加入S
    D[v0]=0; // 源点到源点的距离为0
    /*-----初始化结束,开始主循环,每次求得v0到某个顶点v的最短路径,将v加到S集-----*/
    for(i=1;i<n;++i) // 对其余n-1个顶点,依次进行计算
    {
        min=MaxInt;
        for(w=0;w<n;++w)
            if(!S[w]&&D[w]<min)
                {v=w;min=D[w];} // 选择一条当前的最短路径,终点为v
        S[v]=true; // 将v加入S
        for(w=0;w<n;++w) // 更新从v0出发到集合V-S上所有顶点的最短路径长度
            if(!S[w]&&(D[v]+G.arcs[v][w]<D[w]))
            {
                D[w]=D[v]+G.arcs[v][w]; // 更新D[w]
                Path[w]=v; // 更改w的前驱为v
            } // if
    } // for
}
```

最短路径的主循环共进行 $n - 1$ 次,每次执行的时间是 $O(n)$,所以算法的时间复杂度是 $O(n^2)$ 。如果用带权的邻接表作为有向图的存储结构,则虽然修改 D 的时间可以减少,

但由于在 D 中选择最小分量的时间不变,所以时间复杂度仍为 $O(n^2)$ 。

多源最短路径算法——弗洛伊德(Floyd)算法

求解每一对顶点之间的最短路径有两种方法:其一是分别以图中的每个顶点为源点共调用 n

次迪杰斯特拉算法;其二是采用下面介绍的弗洛伊德(Floyd)算法。两种算法的时间复杂度均

为 $O(n^3)$,但后者形式上较简单。

弗洛伊德算法仍然使用带权的邻接矩阵 $arcs$ 来表示有向网 G ,求从顶点 v_i 到 v_j 的最短路径。

算法的实现要引入以下辅助的数据结构。

(1) 二维数组 $Path[i][j]$: 最短路径上顶点 v_j 的前一顶点的序号。

(2) 二维数组 $D[i][j]$: 记录顶点 v_i 和 v_j 之间的最短路径长度。

【算法步骤】

将 v_i 到 v_j 的最短路径长度初始化,即 $D[i][j] = G.arcs[i][j]$,然后进行 n 次比较和更新。

① 在 v_i 和 v_j 间加入顶点 v_0 ,比较 (v_i, v_j) 和 (v_i, v_0, v_j) 的路径长度,取其中较短者作为 v_i 到 v_j 的中间顶点序号不大于 0 的最短路径。

② 在 v_i 和 v_j 间加入顶点 v_1 ,得到 $(v_i, \dots, v_1)$ 和 $(v_1, \dots, v_j)$,其中 $(v_i, \dots, v_1)$ 是从 v_i 到 v_1 的且中间顶点的序号不大于 0 的最短路径, $(v_1, \dots, v_j)$ 是从 v_1 到 v_j 的且中间顶点的序号不大于 0 的最短路径,这两

条路径已在上一步中求出。比较 $(v_i, \dots, v_l, \dots, v_j)$ 与上一步求出的 v_i 到 v_j 的中间顶点序号不大于 0 的最短路径，取其中较短者作为 v_i 到 v_j 的中间顶点序号不大于 1 的最短路径。

③ 依次类推，在 v_i 和 v_j 间加入顶点 v_k ，若 $(v_i, \dots, v_k)$ 和 $(v_k, \dots, v_j)$ 分别是 v_i 到 v_k 和从 v_k 到 v_j 的中间

顶点的序号不大于 $k-1$ 的最短路径，则将 $(v_i, \dots, v_k, \dots, v_j)$ 和已经得到的从 v_i 到 v_j 且中间顶点序号不大

于 $k-1$ 的最短路径相比较，其长度较短者便是从 v_i 到 v_j 的中间顶点的序号不大于 k 的最短路径。这

样，经过 n 次比较后，最后求得的必是从 v_i 到 v_j 的最短路径。按此方法，可以同时求得各对顶点间的最短路径。

根据上述求解过程，图中的所有顶点对 v_i 和 v_j 间的最短路径长度对应一个 n 阶方阵 D 。在上述

$n+1$ 步中， D 的值不断变化，对应一个 n 阶方阵序列。

n 阶方阵序列可定义为：

$D(-1), D(0), D(1), \dots, D(k), \dots, D(n-1)$

其中，

$D(-1)[i][j] = G.\text{arcs}[i][j]$

$D(k)[i][j] = \min\{D(k-1)[i][j], D(k-1)[i][k] + D(k-1)[k][j]\}$

$0 \leq k \leq n-1$

显然， $D(1)[i][j]$ 是从 v_i 到 v_j 的且中间顶点的序号不大于 1 的最短路径的长度； $D(k)[i][j]$ 是从 v_i 到 v_j 的中间顶点的序号不大于 k 的最短路径的长度； $D(n-1)[i][j]$

就是从 v_i 到 v_j 的最短路径的长度。

```
void ShortestPath_Floyd(AMGraph G)
{ // 用弗洛伊德算法求有向网 G 中各对顶点 i 和 j 之间的最短路径
    for (i=0; i<G.vexnum; ++i) // 各对顶点之间初始已知路径及距离
        for (j=0; j<G.vexnum; ++j)
        {
            D[i][j]=G.arcs[i][j];
            if (D[i][j]<MaxInt && i!=j) Path[i][j]=i; // 如果 i 和 j 之间有弧，则将 j 的前驱置为 i
            else Path[i][j]=-1; // 如果 i 和 j 之间无弧，则将 j 的前驱置为 -1
        } // for
    for (k=0; k<G.vexnum; ++k)
        for (i=0; i<G.vexnum; ++i)
            for (j=0; j<G.vexnum; ++j)
                if (D[i][k]+D[k][j]<D[i][j]) // 从 i 经 k 到 j 的一条路径更短
                {
                    D[i][j]=D[i][k]+D[k][j]; // 更新 D[i][j]
                    Path[i][j]=Path[k][j]; // 更改 j 的前驱为 k
                } // if
}
```

算法 图的应用拓扑排序

- (1) 在有向图中选一个无前驱的顶点且输出它。
- (2) 从图中删除该顶点和所有以它为尾的弧。
- (3) 重复 (1) 和 (2)，直至不存在无前驱的顶点。
- (4) 若此时输出的顶点数小于有向图中的顶点数，则说明有向图中存在环，否则输出的顶点序列即一个拓扑序列

【算法步骤】

- ① 求出各顶点的入度并存入数组 `indegree[i]` 中，使入度为 0 的顶点入栈。
- ② 只要栈不空，则重复以下操作：
使栈顶顶点 v_i 出栈并保存在拓扑序列数组 `topo` 中；
对顶点 v_i 的每个邻接点 v_k 的入度减 1，如果 v_k 的入度变为 0，则使 v_k 入栈。
- ③ 如果输出顶点个数少于 AOV-网的顶点个数，则网中存在有向环，无法进行拓扑排序，否则拓扑排序成功。

```
Status TopologicalSort(ALGraph G,int topo[])
{ // 有向图 G 采用邻接表作为存储结构
  // 若 G 无回路，则生成 G 的一个拓扑序列 topo 并返回 OK，否则 ERROR
  FindInDegree(G,indegree); // 求出各顶点的入度并存入数组 indegree 中
  InitStack(S); // 栈 S 初始化为空
  for(i=0;i<G.vexnum;++i)
    if(!indegree[i]) Push(S,i); // 入度为 0 者进栈
  m=0; // 对输出顶点计数，初始为 0
  while(!StackEmpty(S)) // 栈 S 非空
  {
    Pop(S,i); // 使栈顶顶点  $v_i$  出栈
    topo[m]=i; // 将顶点  $v_i$  保存在拓扑序列数组 topo 中
    ++m; // 对输出顶点计数
    p=G.vertices[i].firstarc; // p 指向顶点  $v_i$  的第一个邻接点
    while(p!=NULL)
    {
      k=p->adjvex; //  $v_k$  为  $v_i$  的邻接点
      --indegree[k]; //  $v_i$  的每个邻接点的入度减 1
      if(indegree[k]==0) Push(S,k); // 若入度减为 0，则入栈
      p=p->nextarc; // p 指向顶点  $v_i$  下一个邻接节点
    }
    //while
  }
  //while
  if(m<G.vexnum) return ERROR; // 该有向图有回路
  else return OK;
}
```

建立求各顶点入度的时间复杂度为 $O(e)$ ；建立零入度顶点栈的时间复杂度为 $O(n)$ ；在拓扑排序过程中，若有向图无环，则每个顶点进一次栈，出一次栈，入度减 1 的操作在循环中总共执行 e 次，所以，总的时间复杂度为 $O(n + e)$ 。

算法 图的应用——关键路径

此算法难度较大感兴趣的同学可以下去仔细了解

【算法步骤】

- ① 调用拓扑排序算法，使拓扑序列保存在 `topo` 中。
- ② 将每个事件的最早发生时间 $ve[i]$ 初始化为 0，即 $ve[i] = 0$ 。
- ③ 根据 `topo` 中的值，按从前向后的拓扑次序，依次求每个事件的最早发生时间，循环几

次，执行以下操作：

取得拓扑序列中的顶点序号 k ， $k = topo[i]$ ；

用指针 p 依次指向 k 的每个邻接顶点，取得每个邻接顶点的序号 $j = p$

$\rightarrow adjvex$ ，依次更新

顶点 j 的最早发生时间 $ve[j]$ ：

$if(ve[j] < ve[k] + p \rightarrow weight)$

$ve[j] = ve[k] + p \rightarrow weight;$

- ④ 将每个事件的最迟发生时间 $vl[i]$ 初始化为汇点的最早发生时间，即 $vl[i] = ve[n-1]$ 。

- ⑤ 根据 `topo` 中的值，按从后向前的逆拓扑次序，依次求每个事件的最迟发生时间，循环 n

次，执行以下操作：

取得拓扑序列中的顶点序号 k ， $k = topo[i]$ ；

用指针 p 依次指向 k 的每个邻接顶点，取得每个邻接顶点的序号 $j = p$

$\rightarrow adjvex$ ，依次根据 k 的邻接点，更新 k 的最迟发生时间 $vl[k]$ ：

$if(vl[k] > vl[j] - p \rightarrow weight) \quad vl[k] = vl[j] - p \rightarrow weight;$

- ⑥ 判断某一活动是否为关键活动，循环 n 次，执行以下操作：对于每个顶点 vi

，用指针 p 依次指向 vi

的每个邻接顶点，取得每个邻接顶点的序号

$j = p \rightarrow adjvex$ ，分别计算活动 $\langle vi, vj \rangle$ 的最早和最迟开始时间 e 和 l ：

$e = ve[i]; \quad l = vl[j] - p \rightarrow weight;$

如果 e 和 l 相等，则活动 $\langle vi, vj \rangle$ 为关键活动，输出弧 $\langle vi, vj \rangle$ 。


```

Status CriticalPath(ALGraph G)
{
    //G为邻接表存储的有向网, 输出G的各项关键活动
    if(!TopologicalOrder(G, topo)) return ERROR;
    //调用拓扑排序算法, 使拓扑序列保存在topo中, 若调用失败, 则存在有向环, 返回ERROR
    n=G.vexnum; //n为顶点个数
    for(i=0; i<n; i++) //给每个事件的最早发生时间置初值0
        ve[i]=0;
    /*-----按拓扑次序求每个事件的最早发生时间-----*/
    for(i=0; i<n; i++)
    {
        k=topo[i]; //取得拓扑序列中的顶点序号k
        p=G.vertices[k].firstarc; //p指向k的第一个邻接顶点
        while(p!=NULL)
        {
            //依次更新k的所有邻接顶点的最早发生时间
            j=p->adjvex; //j为邻接顶点的序号
            if(ve[j]<ve[k]+p->weight) //更新顶点j的最早发生时间ve[j]
                ve[j]=ve[k]+p->weight;
            p=p->nextarc; //p指向k的下一个邻接顶点
        } //while
    } //for
    for(i=0; i<n; i++) //给每个事件的最迟发生时间置初值ve[n-1]
        vl[i]=ve[n-1];
    /*-----按逆拓扑次序求每个事件的最迟发生时间-----*/
    for(i=n-1; i>=0; i--)
    {
        k=topo[i]; //取得拓扑序列中的顶点序号k
        p=G.vertices[k].firstarc; //p指向k的第一个邻接顶点
        while(p!=NULL) //根据k的邻接点, 更新k的最迟发生时间
        {
            j=p->adjvex; //j为邻接顶点的序号
            if(vl[k]>vl[j]-p->weight) //更新顶点k的最迟发生时间vl[k]
                vl[k]=vl[j]-p->weight;
            p=p->nextarc; //p指向k的下一个邻接顶点
        } //while
    } //for
    /*-----判断每一活动是否为关键活动-----*/
    for(i=0; i<n; i++) //每次循环针对vi为活动开始点的所有活动
    {
        p=G.vertices[i].firstarc; //p指向i的第一个邻接顶点
        while(p!=NULL)
        {
            j=p->adjvex; //j为i的邻接顶点的序号
            e=ve[i]; //计算活动<vi, vj>的最早开始时间
            l=vl[j]-p->weight; //计算活动<vi, vj>的最迟开始时间
            if(e==l) //若为关键活动, 则输出<vi, vj>
                cout<<G.vertices[i].data<<G.vertices[j].data;
            p=p->nextarc; //p指向i的下一个邻接顶点
        } //while
    } //for
}

```

第七章 查找

算法 顺序查找

7.2.1 顺序查找

```
// 顺序查找（下标从0开始、无哨兵、无结构体）
int Search_Seq(int A[], int n, int key) {
    // 遍历数组所有元素（下标0 ~ n-1）
    for (int i = 0; i < n; i++) {
        if (A[i] == key) { // 找到目标值，返回对应下标
            return i;
        }
    }
    return -1; // 遍历结束未找到，返回-1（与折半查找返回规则一致）
}
```

•时间复杂度：O(n)

查找成功时的平均查找长度，设表中各记录查找概率相等

$$ASL_s(n) = (1+2+...+n)/n = (n+1)/2$$

•空间复杂度：一个辅助空间——O(1);

顺序查找的特点

优点：算法简单，逻辑次序无要求，且不同存储结构均适用。

缺点：ASL太长，时间效率太低。

算法 折半查找

折半查找算法：（非递归算法）

- 设表长为n，low、high和mid分别指向待查元素所在区间的上界、下界和中点，key为给定的要查找的值：
- 初始时，令low=1，high=n，mid= $\lfloor (low+high)/2 \rfloor$
- 让k与mid指向的记录比较
 - 若key==R[mid].key，查找成功
 - 若key<R[mid].key，则 high=mid-1
 - 若key>R[mid].key，则 low=mid+1
- 重复上述操作，直至low>high时，查找失败

【算法7.3】折半查找

```
int Search_Bin ( SSTable ST, KeyType key ) {  
    low = 1; high = ST.length; // 置区间初值  
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (ST.R[mid].key == key) return mid; // 找到待查元素  
        else if (key < ST.R[mid].key) // 缩小查找区间  
            high = mid - 1; // 继续在前半区间进行查找  
        else low = mid + 1; // 继续在后半区间进行查找  
    }  
    return 0; // 顺序表中不存在待查元素  
} // Search_Bin
```

补充【算法7.3】折半查找——递归算法

```
int Search_Bin (SSTable ST, keyType key, int low, int high) {  
    if (low > high) return 0; // 查找不到时返回0  
    mid = (low + high) / 2;  
    if (key == ST.elem[mid].key) return mid;  
    else if (key < ST.elem[mid].key)  
        ..... // 递归, 在前半区间进行查找  
    else ..... // 递归, 在后半区间进行查找  
}
```

折半查找**优点**：效率比顺序查找高。

折半查找**缺点**：只适用于**有序表**，且限于**顺序存储结构**（对线性链表无效）

折半查找的时间复杂度为 $O(\log_2 n)$ 。想了解为什么可以去查看折半判定树

算法 二叉排序树递归查找

【算法步骤】

- ① 若二叉排序树为空，则查找失败，返回空指针。
- ② 若二叉排序树非空，将给定值 key 与根节点的关键字 $T \rightarrow key$ 进行比较：

若 key 等于 $T \rightarrow key$ ，则查找成功，返回根节点地址；

若 key 小于 $T \rightarrow key$ ，则递归查找左子树；

若 key 大于 $T \rightarrow key$ ，则递归查找右子树。

```
// 在二叉排序树T中查找key: 找到返回节点指针, 没找到返回NULL
BSTNode* SearchBST(BSTNode* T, int key) {
    // 终止条件: 树空 (没找到) 或 找到目标节点
    if (T == NULL || T->key == key) {
        return T;
    }
    // 目标更小 -> 递归查左子树
    else if (key < T->key) {
        return SearchBST(T->lchild, key);
    }
    // 目标更大 -> 递归查右子树
    else {
        return SearchBST(T->rchild, key);
    }
}
```

写出二叉排序树查找的非递归算法

```
key)
关键字等于key的数据元素
指针, 否则返回空指针
n T; // 查找结束
SearchBST(T->lchild, key); // 在左子树中继续查找
key); // 在右子树中继续查找
```

平均查找长度仍然和 $\log_2 n$ 是同数量级的。

算法 二叉排序树的插入

【算法步骤】

- ① 若二叉排序树为空，则将待插入节点 $*S$ 作为根节点插入空树。
- ② 若二叉排序树非空，则将 key 与根节点的关键字 $T \rightarrow data.key$ 进行比较：

若 key 小于 $T \rightarrow data.key$ ，则将 $*S$ 插入左子树；

```
// 向二叉排序树T中插入键值key (重复键不插入)
void InsertBST(BSTNode* &T, int key) {
    // 树空: 创建新节点作为根节点
    if (T == NULL) {
        BSTNode* newNode = new BSTNode;
        newNode->key = key;
        newNode->lchild = newNode->rchild = NULL;
        T = newNode;
    }
    // 键值更小: 递归插入左子树
    else if (key < T->key) {
        InsertBST(T->lchild, key);
    }
    // 键值更大: 递归插入右子树
    else if (key > T->key) {
        InsertBST(T->rchild, key);
    }
    // 键值相等: 不插入 (二叉排序树不允许重复键)
}
```

```
be e)
key的数据元素时, 则插入该元素
// 找到插入位置, 递归结束
// 生成新节点 *s
// 新节点 *s 的数据域置为 e
// 新节点 *s 作为叶子节点
// 把新节点 *s 链接到已找到的插入位置
// 将 *s 插入左子树
// 将 *s 插入右子树
```

若 key 大于 $T \rightarrow data.key$ ，则将 $*S$ 插入右子树。

二叉排序树插入的基本过程是查找，所以时间复杂度同查找一样，是 $O(\log_2 n)$ 。

算法 二叉排序树的删除

【算法步骤】

首先从二叉排序树的根节点开始查找关键字为 key 的待删节点，如果树中不存在此节点，则

不做任何操作；否则，假设被删节点为 *p（指向节点的指针为 p），其双亲节点为 *f（指向节点

的指针为 f），PL 和 PR 分别表示其左子树和右子树[见图 7.9（a）]。

不失一般性，可设 *p 是 *f 的左孩子（右孩子情况类似）。下面分 3 种情况进行讨论。

（1）若 *p 节点为叶子节点，即 PL 和 PR 均为空树。由于删去叶子节点不破坏整棵树的结构，

因此只需修改其双亲节点的指针即可：

```
f->lchild = NULL;
```

（2）若 *p 节点只有左子树 PL 或者只有右子树 PR，此时只要令 PL 或 PR 直接成为其双亲节点 *f 的左子树即可：

```
f->lchild = p->lchild; (或 f->lchild = p->rchild;)
```

（3）若 *p 节点的左子树和右子树均不空。从图 7.9（b）可知，在删去 *p 节点之前，中序遍

历该二叉树得到的序列为 $\{\dots CLC \dots QLQSLSPPRF \dots\}$ ，在删去 *p 之后，为保持其他元素之间的相

对位置不变，可以有两种处理方法：

① 令 *p 的左子树为 *f 的左子树，而 *p 的右子树为 *s 的右子树，如图 7.9（c）所示。

```
f->lchild = p->lchild; s->rchild = p->rchild;
```

② 令 *p 的直接前驱（或直接后继）替代 *p，然后再从二叉排序树中删去它的直接前驱（或直接后继）。如图 7.9（d）所示，当以直接前驱 *s 替代 *p 时，由于 *s 只有左子树 SL，因此在删去 *s 之后，只要令 SL 为 *s 的双亲 *q 的右子树即可

```

void DeleteBST(BSTree &T,KeyType key)
{ // 从二叉排序树T中删除关键字等于key的节点
    p=T;f=NULL; // 初始化
    /*----- 下面的while循环从根开始查找关键字等于key的节点 *p-----*/
    while(p)
    {
        if(p->data.key==key) break; // 找到关键字等于key的节点 *p, 结束循环
        f=p; // *f 为 *p 的双亲节点
        if(p->data.key>key) p=p->lchild; // 在 *p 的左子树中继续查找
        else p=p->rchild; // 在 *p 的右子树中继续查找
    } //while
    if(!p) return; // 找不到被删节点则返回
    /*---- 考虑3种情况实现p所指子树内部的处理：*p左右子树均不空、无右子树、无左子树 ----*/
    q=p;
    if((p->lchild)&&(p->rchild)) // 被删节点 *p 左右子树均不空
    {
        s=p->lchild;
        while (s->rchild) // 在 *p 的左子树中继续查找其前驱节点，即最右下节点
        {
            q=s; s=s->rchild; // 向右到尽头
        }
        p->data=s->data; // s 指向被删节点的“前驱”
        if(q!=p) q->rchild=s->lchild; // 重接 *q 的右子树
        else q->lchild=s->lchild; // 重接 *q 的左子树
        delete s;
        return;
    } //if
    else if(!p->rchild) // 被删节点 *p 无右子树，只需重接其左子树
    {
        p=p->lchild;
    } //else if
    else if(!p->lchild) // 被删节点 *p 无左子树，只需重接其右子树
    {
        p=p->rchild;
    } //else if
    /*----- 将p所指的子树挂接到其双亲节点 *f 相应的位置 -----*/
    if(!f) T=p; // 被删节点为根节点
    else if(q==f->lchild) f->lchild=p; // 挂接到 *f 的左子树位置
    else f->rchild=p; // 挂接到 *f 的右子树位置
    delete q;
}

```

同二叉排序树插入一样，二叉排序树删除的基本过程也是查找，所以时间复杂度仍是 $O(\log 2n)$ 。

22-11-1

第八章 排序

算法 直接插入排序

【算法步骤】

- ① 设待排序的记录存放在数组 $r[1..n]$ 中, $r[1]$ 是一个有序序列。
- ② 循环 $n-1$ 次, 每次使用顺序查找法, 查找 $r[i]$ ($i = 2, \dots, n$) 在已排好序的序列 $r[1 \dots i-1]$ 中的插入位置, 然后将 $r[i]$ 插入表长为 $i-1$ 的有序序列 $r[1 \dots i-1]$, 直到将 $r[n]$ 插入表长为 $n-1$ 的有序序列 $r[1 \dots n-1]$, 最后得到一个表长为 n 的有序序列

直接插入排序算法

```
void InsertSort( SqList &L ) {  
    int i, j;  
    for (i=2; i<=L.length; ++i) {  
        if (L.r[i].key < L.r[i-1].key) { // 若 "<", 需将 L.r[i] 插入有序子表  
            L.r[0] = L.r[i]; // 复制为哨兵  
            for (j=i-1; L.r[0].key < L.r[j].key; --j) {  
                L.r[j+1] = L.r[j]; // 记录后移  
            }  
            L.r[j+1] = L.r[0]; // 插入到正确位置  
        }  
    }  
}
```

从时间来看, 排序的基本操作为比较两个关键字的大小和移动记录。

直接插入排序的时间复杂度为 $O(n^2)$ 。

直接插入排序只需要一个记录的辅助空间 $r[0]$, 所以空间复杂度为 $O(1)$ 。

【算法特点】

- (1) 稳定排序。
- (2) 算法简便, 且容易实现。
- (3) 也适用于链式存储结构, 只是在单链表上无须移动记录, 只需修改相应的指针。
- (4) 更适用于初始记录基本有序 (正序) 的情况, 当初始记录无序、 n 较大时, 此算法时间复杂度较高, 不宜采用。

22-pp1-

算法 折半插入排序

【算法步骤】

① 设待排序的记录存放在数组 $r[1 \cdots n]$ 中， $r[1]$ 是一个有序序列。

折半插入排序

② 循环 $n-1$ 次，每次使用折半查找法，查找 $r[i]$ ($i = 2, \dots, n$) 在已排好序的序列 $r[1..i-1]$ 中的插入位置，然后将 $r[i]$ 插入表长为 $i-1$ 的有序序列 $r[1 \cdots i-1]$ ，

直到将 $r[n]$ 插入表长为 $n-1$ 的有序序列 $r[1 \cdots n-1]$ ，最后得到一个表长为 n 的有序序列。

```
void BInsertSort ( SqList &L ) {  
    for ( i = 2; i <= L.length; ++i ) { //依次插入第2~第n个元素  
        L.r[0] = L.r[i]; //当前插入元素存到“哨兵”位置  
        low = 1; high = i-1; //采用二分查找法查找插入位置  
        while ( low <= high ) {  
            mid = ( low + high ) / 2;  
            if ( L.r[0].key < L.r[mid].key ) high = mid - 1;  
            else low = mid + 1;  
        } //循环结束, high+1则为插入位置  
        for ( j=i-1; j>=high+1; --j ) L.r[j+1] = L.r[j]; //移动元素  
        L.r[high+1] = L.r[0]; //插入到正确位置  
    }  
} // BInsertSort
```

从时间上比较，折半查找比顺序查找快，所以就平均性能来说，折半插入排序优于直接插入排序。在平均情况下，折半插入排序仅减少了关键字间的比较次数，而记录的移动次数不变。因此，折半插入排序的时间复杂度仍为 $O(n^2)$ 。

折半插入排序所需附加存储空间和直接插入排序相同，只需要一个记录的辅助空间 $r[0]$ ，所以空间复杂度为 $O(1)$ 。

【算法特点】

- (1) 稳定排序。
- (2) 因为要进行折半查找，所以只能用于顺序结构，不能用于链式结构。
- (3) 适合初始记录无序、 n 较大的情况。

22-PP1-

算法 希尔排序

希尔排序 (Shell's Sort) 又称“缩小增量排序” (Diminishing Increment Sort), 是插入排序的一种。当待排序的记录个数较少且待排序序列的关键字基本有序时, 直接插入排序效率较高。希尔排序基于以上两点, 从“减少记录个数”和“序列基本有序”两个方面对直接插入排序进行了改进。

【算法步骤】

希尔排序实质上是采用分组插入的方法, 先将整个待排序记录序列分割成几组, 从而减少参与直接插入排序的数据量, 对每组分别进行直接插入排序, 然后增加每组的数据量, 重新分组。这样当经过几次分组排序后, 整个序列中的记录“基本有序”时, 再对全体记录进行一次直接插入排序。

希尔对记录的分组, 不是简单地“逐段分割”, 而是将相隔某个“增量”的记录分成一组。

① 第一趟取增量 d_1 ($d_1 < n$) 把全部记录分成 d_1 个组, 所有间隔为 d_1 的记录分在同一组, 在各个组中进行直接插入排序。

② 第二趟取增量 d_2 ($d_2 < d_1$), 重复上述的分组和排序。

③ 依次类推, 直到所取的增量 $d_t = 1$ ($d_t < d_{t-1} < \dots < d_2 < d_1$), 所有记录在同一组中进行直接插入排序为止。

```
void ShellInsert(SqList &L, int dk)
{
    // 对顺序表L进行一趟增量是dk的希尔插入排序
    for(i=dk+1; i<=L.length; ++i)
    {
        if(L.r[i].key<L.r[i-dk].key) // 需将L.r[i] 插入有序增量子表
        {
            L.r[0]=L.r[i]; // 暂存在r[0] 中
            for(j=i-dk; j>0&& L.r[0].key<L.r[j].key; j-=dk)
                L.r[j+dk]=L.r[j]; // 记录后移, 直到找到插入位置
            L.r[j+dk]=L.r[0]; // 将r[0] 即原r[i], 插入正确位置
        } // if
    }
}

void ShellSort(SqList &L, int dt[], int t)
{
    // 按增量序列dt[0..t-1] 对顺序表L进行t趟希尔排序
    for(k=0; k<t; ++k)
        ShellInsert(L, dt[k]); // 一趟增量为dt[t] 的希尔插入排序
}
```

到目前为止尚未有人求得一种最好的增量序列, 但大量的研究已得出一些局

部的结论有人指出, 当增量序列为 $dt[k]=2^{t-k+1}-1$ 时, 希尔排序的时间复杂度为 $O(n^{3/2})$

当 n 在某个特定范围内, 希尔排序所需的比较和移动次数约为 $n^{1.3}$, 当 $n \rightarrow \infty$ 时, 比较和移动次数可减少到 $n(\log_2 n)^2$ 。

从空间来看, 希尔排序和前面两种排序方法一样, 也只需要一个辅助空间 $r[0]$, 空间复杂度为 $O(1)$ 。

【算法特点】

- (1) 记录跳跃式地移动导致排序方法是不稳定的。
- (2) 只能用于顺序结构，不能用于链式结构。
- (3) 增量序列可以有各种取法，但应该使增量序列中的值没有除 1 之外的公因子，并且最后一个增量值必须等于 1。
- (4) 记录总的比较次数和移动次数都比直接插入排序的要少， n 越大时，效果越明显。所以适合初始记录无序、 n 较大时的情况。

算法 X 冒泡排序

【算法步骤】

- ① 设待排序的记录存放在数组 $r[1 \cdots n]$ 中。首先将第一个记录的关键字和第二个记录的关键字进行比较，若为逆序（即 $L.r[1].key > L.r[2].key$ ），则交换两个记录。然后比较第二个记录和第三个记录的关键字。依次类推，直至第 $n-1$ 个记录和第 n 个记录的关键字进行过比较为止。上述过程称作第一趟起泡排序，其结果使得关键字最大的记录被安置到最后一个记录的位置上。
- ② 然后进行第二趟起泡排序，对前 $n-1$ 个记录进行同样的操作，其结果是使关键字次大的记录被安置到第 $n-1$ 个记录的位置上。
- ③ 重复上述比较和交换过程，第 i 趟是从 $L.r[1]$ 到 $L.r[n-i+1]$ 依次比较相邻两个记录的关键字，并在“逆序”时交换相邻记录，其结果是这 $n-i+1$ 个记录中关键字最大的记录被交换到第 $n-i+1$ 的位置上。直到在某一趟排序过程中没有进行过交换记录的操作，说明序列已全部达到排序要求，则完成排序。

```
void BubbleSort(ElemType A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool flag = false; // 表示本趟冒泡是否发生交换的标志
        for (int j = n - 1; j > i; j--) // 一趟冒泡过程
            if (A[j - 1] > A[j]) { // 若为逆序
                swap(A[j - 1], A[j]); // 使用封装的 swap 函数交换①
                flag = true;
            }
        if (flag == false)
            return; // 本趟遍历后没有发生交换，说明表已经有序
    }
}
```

在平均情况下，冒泡排序关键字的比较次数和记录移动次数分别约为 $n^{2/4}$ 和 $3^{n^{2/4}}$ ，时间复杂度为 $O(n^2)$ 。

冒泡排序只有在两个记录交换位置时需要一个辅助空间用于暂存记录，所以空间复杂度为 $O(1)$ 。

【算法特点】

- (1) 稳定排序。
- (2) 可用于链式存储结构。
- (3) 移动记录次数较多，算法的平均时间性能比直接插入排序的差。当初始记录无序、 n 较大时，此算法不宜采用。

算法 快速排序

快速排序 (Quick Sort) 是由冒泡排序改进而得的。在冒泡排序过程中，只对相邻的两个记录进行比较，因此每次交换两个相邻记录时只能消除一个逆序排列。

基本思想: 通过一趟排序，将待排序记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录进行排序，以达到整个序列有序

【算法步骤】

在待排序的 n 个记录中任取一个记录 (通常取第一个记录) 作为枢轴 (或支点)，设其关键字为 pivotkey 。经过一趟排序后，把所有关键字小于 pivotkey 的记录交换到前面，把所有关键字大于 pivotkey 的记录交换到后面，结果将待排序记录分成两个子表，最后将枢轴放置在分界处的位置。然后，分别对左、右子表重复上述过程，直至每一子表只有一个记录时，排序完成。

一趟快速排序的具体步骤如下：

① 选择待排序表中的第一个记录作为枢轴，将枢轴记录暂存在 $r[0]$ 的位置上。附设两个指针 low 和 high ，初始时分别指向表的下界和上界 (第一趟时， $\text{low} = 1$; $\text{high} = L.\text{length}$)。

② 从表的最右侧位置依次向左搜索，找到第一个关键字小于枢轴关键字 pivotkey 的记录，将其移到 low 处。具体操作是：当 $\text{low} < \text{high}$ 时，若 high 所指记录的关键字大于等于 pivotkey ，则向左移动指针 high (执行操作 $--\text{high}$)，否则将 high 所指记录与枢轴记录交换。

③ 然后从表的最左侧位置，依次向右搜索找到第一个关键字大于 pivotkey 的记录和枢轴记

录交换。具体操作是：当 $\text{low} < \text{high}$ 时，若 low 所指记录的关键字小于等于 pivotkey ，则向右移动指针 low (执行操作 $++\text{low}$)，否则将 low 所指记录与枢轴记录交换。

④ 重复步骤②和步骤③，直至 low 与 high 相等为止。此时 low 或 high 的位置即枢轴在此趟排序中的最终位置，原表被分成两个子表。在上述过程中，记录的交换都是与枢轴之间发生的，每次交换都要移动 3 次记录，可以先将枢轴记录暂存在 $r[0]$ 的位置上，排序过程中只移动要与枢轴交换的记录，即只进行 $r[\text{low}]$ 或 $r[\text{high}]$ 的单向移动，直至一趟排序结束后再将枢轴记录移至正确位置上。

```

int Partition(SqList &L, int low, int high)
{
    // 对顺序表L中的子表r[low..high] 进行一趟排序, 返回枢轴位置
    L.r[0]=L.r[low];           // 用子表的第一个记录作为枢轴记录
    pivotkey=L.r[low].key;      // 枢轴记录关键字保存在pivotkey中
    while(low<high)             // 从表的两端交替地向中间查找
    {
        while(low<high&&L.r[high].key>=pivotkey) --high;
        L.r[low]=L.r[high];     // 将比枢轴记录小的记录移到低端
        while(low<high&&L.r[low].key<=pivotkey) ++low;
        L.r[high]=L.r[low];     // 将比枢轴记录大的记录移到高端
    }                           // while
    L.r[low]=L.r[0];            // 枢轴记录到位
    return low;                // 返回枢轴位置
}

void QSort(SqList &L,int low,int high)
{
    // 调用前置初值: low=1; high=L.length;
    // 对顺序表L中的子表L.r[low..high] 进行快速排序
    if(low<high) {              // 长度大于1
        pivotloc=Partition(L, low, high); // 将L.r[low..high] 一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1);      // 对左子表递归排序
        QSort(L, pivotloc+1, high);     // 对右子表递归排序
    }
}

void QuickSort(SqList &L)
{
    // 对顺序表L进行快速排序
    QSort(L,1,L.length);
}

```

从快速排序算法的递归树可知, 快速排序的趟数取决于递归树的深度理论上可以证明, 平均情况下, 快速排序的时间复杂度为 $O(n\log_2 n)$ 。

快速排序是递归的, 执行时需要有一个栈来存放相应的数据。最大递归调用次数与递归树的深度一致, 所以最好情况下的空间复杂度为 $O(\log_2 n)$, 最坏情况下为 $O(n)$ 。

【算法特点】

- (1) 记录非顺次的移动导致排序方法是不稳定的。
- (2) 排序过程中需要定位表的下界和上界, 所以适合用于顺序结构, 很难用于链式结构。
- (3) 当 n 较大时, 在平均情况下快速排序是所有内部排序方法中速度最快的一种, 所以其适合初始记录无序、 n 较大的情况。

22-pp1-

算法 简单选择排序

基本思想：在待排序的数据中选出最大（小）的元素放在其最终的位置。

基本操作：

1. 首先通过 $n-1$ 次关键字比较，从 n 个记录中找出关键字最小的记录，将它与第一个记录交换
2. 再通过 $n-2$ 次比较，从剩余的 $n-1$ 个记录中找出关键字次小的记录，将它与第二个记录交换
3. 重复上述操作，共进行 $n-1$ 趟排序后，排序结束

简单选择排序算法

```
void SelectSort(SqList &L) {  
    for (i=1; i<L.length; ++i) {  
        k=i;  
        for (j=i+1; j<=L.length; j++)  
            if ( L.r[j].key < L.r[k].key) k=j; //记录最小值位置  
        if (k!=i) L.r[i] ↔ L.r[k]; //交换  
    }  
}
```

简单选择排序的时间复杂度是 $O(n^2)$ 。

同冒泡排序一样，只有在两个记录交换时需要一个辅助空间，所以空间复杂度为 $O(1)$ 。

【算法特点】

(1) 就选择排序方法本身来讲，它是一种稳定的排序方法，但图 8.6 所表现出来的现象是不稳定的，这是因为上述实现选择排序的算法采用“交换记录”的策略所造成的，改变这个策略，可以写出不产生“不稳定现象”的选择排序算法。

(2) 可用于链式存储结构。

(3) 移动记录次数较少，当每一记录占用的空间较多时，此方法比直接插入排序快。

算法 堆排序

堆排序 (Heap Sort) 是一种树形选择排序, 在排序过程中, 将待排序的记录 $r[1..n]$ 看成一棵完全二叉树的顺序存储结构, 利用完全二叉树中双亲节点和孩子节点之间的内在关系, 在当前无序的序列中选择关键字最大 (或最小) 的记录。

下面讨论用大根堆进行排序, 堆排序的步骤如下:

① 按堆的定义将待排序序列 $r[1..n]$ 调整为大根堆 (这个过程称为建初堆), 交换 $r[1]$ 和 $r[n]$, 则 $r[n]$ 为关键字最大的记录。

② 将 $r[1..n-1]$ 重新调整为堆, 交换 $r[1]$ 和 $r[n-1]$, 则 $r[n-1]$ 为关键字次大的记录。

③ 循环 $n-1$ 次, 直到交换 $r[1]$ 和 $r[2]$ 为止, 得到一个非递减的有序序列 $r[1..n]$ 。

调整堆算法

```
void HeapAdjust(SqList &L, int s, int m)
{ // 假设  $r[s+1..m]$  已经是堆, 将  $r[s..m]$  调整为以  $r[s]$  为根的大根堆
    rc = L.r[s];
    for (j = 2*s; j <= m; j *= 2) // 沿 key 较大的孩子节点向下筛选
    {
        if (j < m && L.r[j].key < L.r[j+1].key) ++j; // j 为 key 较大的记录的下标
        if (rc.key >= L.r[j].key) break; // rc 应插入在位置 s 上
        L.r[s] = L.r[j]; s = j;
    } // for
    L.r[s] = rc; // 插入
}
```

建立堆算法

```
void CreatHeap(SqList &L)
{ // 把无序序列  $L.r[1..n]$  建成大根堆
    n = L.length;
    for (i = n/2; i > 0; --i) // 反复调用 HeapAdjust
        HeapAdjust(L, i, n);
}
```

堆排序算法

```
void HeapSort(SqList &L)
{ // 对顺序表 L 进行堆排序
    CreatHeap(L); // 把无序序列  $L.r[1..L.length]$  建成大根堆
    for (i = L.length; i > 1; --i)
    {
        x = L.r[1]; // 将堆顶记录和当前未经排序子序列  $L.r[1..i]$  中最后一个记录互换
        L.r[1] = L.r[i];
        L.r[i] = x;
        HeapAdjust(L, 1, i-1); // 将  $L.r[1..i-1]$  重新调整为大根堆
    } // for
}
```

时间复杂度

堆排序的运行时间主要耗费在建初堆和调整堆时进行的反复筛选上。

堆排序在最坏的情况下, 其时间复杂度也为 $O(n\log_2 n)$ 实验研究表明, 堆

可以看看是否要记忆一个函数

2-路归并排序

排序的平均性能接近于最坏性能

仅需一个记录大小供交换用的辅助存储空间，所以空间复杂度为 $O(1)$ 。

算法 二路归并排序

【算法步骤】

2-路归并排序将 $R[\text{low}..\text{high}]$ 中的记录归并排序后放入 $T[\text{low}..\text{high}]$ 中。
当序列长度等于 1 时，递归结束，否则：

- ① 将当前序列一分为二，求出分裂点 $\text{mid} = (\text{low} + \text{high}) / 2$ ；
- ② 对子序列 $R[\text{low}..\text{mid}]$ 递归进行归并排序，结果放入 $S[\text{low}..\text{mid}]$ 中；
- ③ 对子序列 $R[\text{mid} + 1..\text{high}]$ 递归进行归并排序，结果放入 $S[\text{mid} + 1..\text{high}]$ 中；

④ 调用算法 Merge，将有序的两个子序列 $S[\text{low}..\text{mid}]$ 和 $S[\text{mid} + 1..\text{high}]$ 归并为一个有序的序列 $T[\text{low}..\text{high}]$ 。

```
void MSort(RedType R[], RedType T[], int low, int high)
{
    // R[low..high] 归并排序后放入 T[low..high] 中
    if (low == high) T[low] = R[low];
    else
    {
        mid = (low + high) / 2; // 将当前序列一分为二，求出分裂点 mid
        MSort(R, S, low, mid); // 对子序列 R[low..mid] 递归进行归并排序，结果放入 S[low..mid]
        MSort(R, S, mid + 1, high); // 对子序列 R[mid+1..high] 递归进行归并排序，结果放入 S[mid+1..high]
        Merge(S, T, low, mid, high); // 将 S[low..mid] 和 S[mid+1..high] 归并到 T[low..high]
    }
}

void MergeSort(SqList &L)
{
    // 对顺序表 L 进行归并排序
    MSort(L.r, L.r, 1, L.length);
}
```

(1) 时间复杂度

当有 n 个记录时，需进行 $\lceil \log_2 n \rceil$ 趟归并排序，每一趟归并的关键字比较次数不超过 n ，元素移动次数都是 n ，因此，归并排序的时间复杂度为 $O(n \log_2 n)$ 。

(2) 空间复杂度

用顺序表实现归并排序时，需要和待排序记录个数相等的辅助存储空间，所以空间复杂度为 $O(n)$ 。

【算法特点】

- (1) 是稳定排序。
- (2) 可用于链式结构，且不需要附加存储空间，但递归实现时仍需要开辟相应的递归工作栈。

表 8.2 各种内部排序方法的比较

排序方法	时间复杂度			空间复杂度	稳定性
	最好情况	最坏情况	平均情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n\log_2 n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序			$O(n^{1.3})$	$O(1)$	不稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n + rd))$	$O(d(n + rd))$	$O(d(n + rd))$	$O(n + rd)$	稳定